

Customer Review Sentiment Analysis using NLP

1. Introduction

In today's digital era, people share their opinions online through reviews, comments, and social media. Understanding these opinions is very important for businesses.

Sentiment Analysis is a Natural Language Processing (NLP) technique used to identify the emotional tone behind a piece of text.

It helps in determining whether a review is:

- Positive 😊
- Negative 😞
- Neutral 😐

This project builds a machine learning model to automatically classify customer reviews into these categories.

2. Objective

The main objectives of this project are:

- To analyze customer reviews using NLP techniques
- To classify text into:
 - Positive
 - Negative
 - Neutral
- To convert text data into numerical form using **TF-IDF**
- To train a machine learning model for prediction

3. Technologies Used

- **Python** – Programming language
- **Pandas** – Data handling
- **Scikit-learn** – Machine learning
- **NLTK / TextBlob** – Text processing (optional)

4. Project Structure

Sentiment_Analysis_Project/

```
|— data/
|   └─ reviews.csv
|— model/
|— sentiment_model.py
|— app.py (optional)
|— README.md
```

Explanation:

- **data/** → Contains dataset
- **model/** → Stores trained model
- **sentiment_model.py** → Core logic
- **app.py** → Optional web app
- **README.md** → Project details

5. Dataset Description

The dataset contains customer reviews and their corresponding sentiment labels.

Format:

reviewId,review,sentiment

Example:

1,This product is amazing,positive

2,I love this item,positive

3,Very bad experience,negative

4,Not worth the money,negative

5,It is okay,neutral

6. Methodology (Step-by-Step Workflow)

Step 1: Load Data

Load dataset using Pandas:

```
df = pd.read_csv("reviews.csv")
```

Step 2: Text Cleaning

Basic preprocessing may include:

- Lowercasing text
- Removing punctuation
- Removing stopwords (optional)

Step 3: Convert Text to Numerical Form

We use **TF-IDF (Term Frequency – Inverse Document Frequency)**:

```
vectorizer = TfidfVectorizer()
```

```
X_vectorized = vectorizer.fit_transform(X)
```

This converts text into numbers so the model can understand it.

Step 4: Train-Test Split

Split data into training and testing:

```
X_train, X_test, y_train, y_test = train_test_split(X_vectorized, y, test_size=0.2)
```

Step 5: Model Training

We use **Logistic Regression**:

```
model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

Step 6: Prediction

```
y_pred = model.predict(X_test)
```

Step 7: Model Evaluation

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

7. Full Working Code

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score


# Load dataset

df = pd.read_csv("reviews.csv")


# Features & Labels

X = df['review']
y = df['sentiment']


# Convert text to numeric

vectorizer = TfidfVectorizer()

X_vectorized = vectorizer.fit_transform(X)


# Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(X_vectorized, y, test_size=0.2)


# Model

model = LogisticRegression()

model.fit(X_train, y_train)
```

```
# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Test with custom input
sample = ["This product is very good"]
sample_vec = vectorizer.transform(sample)

prediction = model.predict(sample_vec)
print("Sentiment:", prediction[0])
```

8. Output

The system produces:

Model Performance:

Accuracy: 0.85 (example)

Sample Prediction:

Input: "This product is very good"

Output: Positive

9. Use Cases

Sentiment Analysis is widely used in:

- E-commerce → Customer review analysis
- Social media → Opinion mining
- Business analytics → Customer feedback
- Brand monitoring → Public perception

10. Challenges

- Handling sarcasm
- Understanding context
- Imbalanced datasets
- Noise in text data

11. Conclusion

This project demonstrates how NLP and machine learning can be used to analyze text data effectively.

Key outcomes:

- Successfully classified reviews into sentiment categories
- Achieved good accuracy using TF-IDF and Logistic Regression
- Built a scalable sentiment analysis system

12. Future Scope

- Use Deep Learning models (LSTM, BERT)
- Real-time sentiment analysis
- Multilingual support
- Integration with web applications